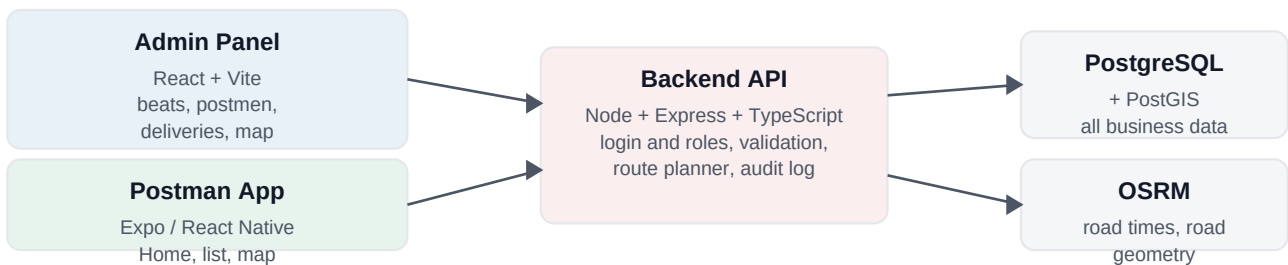


How the system works

Postal Delivery Operations System: the admin panel, the postman app and the backend, in plain words.

1. The big picture

Three programs work together. The **admin panel** is for post-office staff. The **postman app** is for the person on the street. The **backend** sits between them and the database, and is the only place where rules are applied and data is saved. Neither app keeps its own copy of the truth: they ask the backend and show what it says.



2. What each part does

Part	Who uses it	What it does
Admin panel	Administrators	Uploads a beat list, draws and verifies beat territories on a map, imports and assigns deliveries, resolves assignment exceptions, sets whether a delivery photo is required, manages postmen, reads reports and the audit log.
Postman app	Postmen	The delivery list in route order, the route map, Navigate, Start Delivery / Mark Delivered (with a photo when the post office requires one), works offline and syncs later, keeps a history, receives notifications.
Backend	Both apps	Login, roles and passwords, saving to PostgreSQL/PostGIS, matching addresses to beats, planning routes, storing proof photos privately, sending notifications, writing the audit log.
PostgreSQL + PostGIS	Backend only	Stores everything: post offices, beats, their localities and map polygons, postmen, deliveries, history, routes.
OSRM	Backend only	A road-routing service: real driving times between stops and the road line drawn on the map.

3. Life of a parcel

- **Arrives.** An admin adds a delivery, or imports a file. The address is tidied up so that "Farid Nagar, Bhandup West, Mumbai" and "Farid Nagar Bhandup W Mumbai" are recognised as the same place.
- **Matched to a beat - by the beat list first.** Each beat has a list of the localities it covers. The system scores every beat from named evidence (which locality matched, which main area, the pincode) and only assigns when one beat is clearly ahead. If two beats fit equally (for example a road that appears under two beats) or the evidence is weak, the parcel goes to **Assignment Exceptions** with a suggested beat, the confidence and the reason - never a guess.
- **The map point supports the match, it does not replace it.** A point that is only the middle of an area or a pincode can never assign a parcel. A precise (house-level) point can decide when the beat list has nothing to say, but only inside exactly one *verified* territory.
- **Routed.** The backend plans the postman's visiting order (section 5) and the app shows it as numbered stops.
- **Delivered.** The postman taps Start Delivery, then Mark Delivered - taking a photo first if the post office requires one. The backend checks the change is allowed and, where required, that the photo is there; it saves it with history and the audit log. The postman's phone position at the door also teaches the system

where that address is.

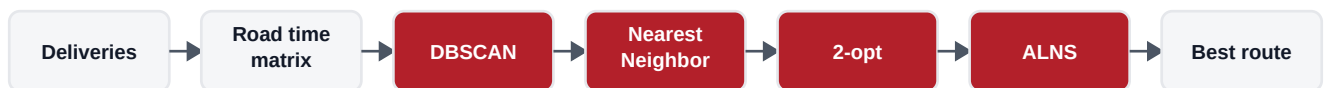
- **Problems.** Recipient unavailable, wrong address and similar outcomes re-plan the route and can raise an exception for the admin.

4. Beats and verification

A beat is an area with one postman: the localities it covers and, optionally, a territory drawn on the map. An admin uploads a beat list (Excel or CSV, one row per beat or per beat and locality) and the panel walks through Upload, Validate, Review, Import: nothing is saved until the last step, every problem is shown and can be downloaded as a report, and no row is silently dropped. Territories are checked when saved (a valid shape, a sensible size and place, no unnoticed overlap with another beat: an overlap must be confirmed) and marked **Verified**, **Needs verification**, **Missing** or **Overlapping**. Only verified territories are used to place a parcel on the map.

5. How the route is chosen

There is exactly one optimizer. It is not a setting, and neither the app nor the admin can change it.



Step	What it does
Road-time matrix	Asks OSRM for the driving time between the start point and every stop. Answers are cached, so repeated plans do not call OSRM again.
DBSCAN	Groups stops that are close <i>by road time</i> into neighbourhoods.
Nearest Neighbor	Builds a first route: neighbourhood by neighbourhood, always going to the closest next stop.
2-opt	Removes crossings by reversing stretches of the route, only when the cost really drops.
ALNS	Repeatedly removes some stops and puts them back in better places (random, worst, related and cluster removal; greedy, regret-2 and regret-3 insertion). Operators that work well are used more often, worse routes are sometimes accepted early on to escape dead ends, and the best route ever seen is what is returned.

Every step is scored with the **same cost**: driving time, plus a penalty for carrying a heavy bag over long legs, plus a penalty for making urgent parcels wait, with time spent at each door included. So a route can only be called better if that number is lower. In the latest test on the 130 real Bhandup West deliveries ALNS improved the 2-opt route by about 15 percent; on tiny routes of a few places it found nothing better, and says so. A route takes well under a second for 100 stops, so it is planned while you wait. The "carrying a heavy bag" part uses real parcel weights when they are known and is switched off when they are not. When a delivery is completed the route is only trimmed; a new delivery or a manual recalculation runs the whole pipeline again.

6. Login, roles and safety

- Everyone signs in with a login; the backend issues short-lived tokens. A postman's identity always comes from the login, never from anything the app sends.
- **First sign-in:** an account made by someone else has a temporary password. The admin panel and the app show only a "Choose a new password" screen, and the backend refuses everything else, until it is changed.
- Roles: Super Admin (all post offices), Admin (their own post office only), Postman (only their own deliveries). Another office's data looks like it does not exist.
- **Delivery photos are private:** stored outside any public folder, readable only by that delivery's postman and the post office's administrators.
- Every important change is written to an audit log: who, what, when. The backend validates every request itself; the apps' buttons are a convenience, not the protection.

7. Data that survives

Beats, polygons, verification, postmen, photos, assignments, deliveries, statuses and routes are all in PostgreSQL. A browser refresh, a logout/login and a backend restart were each tested to leave them unchanged. The only thing kept on the phone is the queue of updates made while offline; it is sent when the connection returns, and if the server has moved on (for example an admin cancelled the parcel) the phone's stale change is not forced through: the postman is told instead.

8. Running and testing it

What	How
Backend	cd backend, then npm run dev (needs PostgreSQL with PostGIS and the .env values). Tests: npm test (unit) and npm run test:integration (a real PostGIS database).
Admin panel	cd frontend, then npm run dev (port 5173). Tests: npm test.
Postman app	cd PostmanApp, then npx expo start --web, or open it on a phone with a development build. Tests: npm test.
Browser tests	e2e/README.md: a real Chrome drives the admin panel and the app against a real backend (45 checks).
Every push	GitHub Actions runs the type checks, ESLint, all the tests above, the build and the browser tests.
Road routing	OSRM_BASE_URL in the backend .env. Without it routes still work but use flagged straight-line estimates.

Known limits: the real beat list is a PDF that is not read (CSV / Excel only); the accuracy figures for the test data are partly circular (the test file lists each delivery's own locality) so the hold-out figure is the fair one; time windows and OTP / signature proof are not built; notifications reach a phone only with a real device build. The public OSRM demo server is for development only.